

Chapter 5 – Methods

1. En esta pregunta vemos acerca del modificador final, para poder entender las opciones es importante recordar que este modificador ya no se puede modificar, la opción A dice que las variables estáticas y de instancia se pueden marcar como final y esto es cierto, la opción B dice que una variable es final solamente si se marca como final, esto no es verdad porque incluso si no se tiene la palabra clave final, la opción C dice que un objeto es marcado como final y no puede ser modificado y es falso porque aquí tiene una trampa de comprensión porque cuando se le coloca el método final lo que no cambia es la referencia al objeto, sin embargo el contenido del objeto si se puede modificar, la D es falsa igual porque dice que las variables var no se pueden declarar junto con el final y la E dice que los datos primitivos una vez nombrados como final no se pueden modificar y es cierto, opciones correctas A y E.
2. En el siguiente fragmento de código pide llenar el espacio en blanco antes del retorno void, es indispensable distinguir que para declarar el método hay que saber que modificador de acceso va, así como su correcta escritura es por eso que inciso B con final y C con private, son las respuestas correctas.
3. Las opciones A y D son correctas porque los especificadores pueden ir en cualquier orden. Por lo tanto las opciones B y C son incorrectas porque cada una tiene dos tipos de retorno y como se vio en el ejercicio pasado no puede haber un método que retorne dos veces. Las opciones E y F son incorrectas porque el tipo de retorno está antes del especificador opcional y del modificador de acceso.
4. Para completar el espacio en blanco se pueden agregar de acuerdo al tipo de carácter que esta después del igual y es inciso A, int, inciso B integer, C long y E double, todo con minúscula.
5. En este ejercicio las opciones correctas son inciso A y C ya que las dos tiene como característica que el método void puede tener opcionalmente una sentencia return siempre y cuando que no tenga un valor de retorno, el inciso B tiene un error de compilación porque null requiere un objeto de referencia como tipo de retorno.
6. Para imprimir el método correcto desmenuzamos que el inciso A es correcto porque está declarando bien tanto el public, como el método y la palabra reservada void, las opciones correctas , el inciso B también es correcta ya que rdtán declarads de manera correcta tanto el método y sus variables. La opción C es incorrecta porque el parámetro varargs no es el último, La opción F es correcta porque no usa ningún parámetro.
7. La opción A no compila porque no pasa el parámetro inicial, La opción B crea un array varargs de tamaño 0, y la opción C crea un array varargs de tamaño 1, La opción D pasa el parámetro inicial más dos más para convertirlo en un array varargs de tamaño 2. La opción F pasa el parámetro inicial más un array de tamaño 2. Las opciones correctas son D y F.
8. De este ejercicio la única opción correcta es D, ya que es una buena práctica establecer todos los campos como privados y todos los métodos como públicos, en primera instancia el inciso A es incorrecta porque el acceso protegido permite todo lo que el acceso de paquete permite y además permite el acceso a las subclases. La opción B es incorrecta porque la clase es pública. Esto significa que otras clases pueden ver la clase. Sin embargo, no pueden llamar a ninguno de los métodos ni leer ninguno de los campos.

9. En este ejercicio se intenta acceder en línea 5 a la variable estática `globalKey` desde la clase `Classroom` pero como no tiene modificador explícito, por lo que usa el acceso por defecto/ `package-private`, entonces genera error de compilación, en la línea 6 se intenta crear una instancia de `Classroom` usando su constructor, el modificador en `classroom` no tiene un modificador explícito, como en la línea anterior y de igual manera genera un error de compilación, las opciones B y C, son correctas, siguiendo con el flujo la línea 7 intenta acceder a la variable de instancia `roomNumber` a través del objeto `room`, pero sale error de compilación porque el modificador `private` restringe el acceso y la línea 9 como está tratando de acceder a un modificador `protected`, incisos A, C, D y F.
10. Lo que entendí de este ejercicio es la aplicación de la JVM cuando la carga la clase `Rope` ejecuta sus componentes estáticos en orden: primero asigna 5 a `LENGTH` y luego el bloque `static`, donde aquí está sobrescribiendo el valor de 10, después al ejecutarse `Chimp`, el método `swing()`, se invoca de manera correcta dos veces, es por eso que la opción correcta es el inciso B.
11. La línea 10 no compila porque no se permite que métodos estáticos llamen a métodos de instancia. Aunque estemos llamando a `play()` como si fuera un método de instancia y exista una instancia, Java sabe que `play()` es realmente un método estático y lo trata como tal. Dado que esta es la única línea que no compila, la opción B es correcta. Si se elimina la línea 10, el código imprime `swing- swing`, haciendo que la opción E sea correcta. No lanza una `NullPointerException` en la línea 17 porque `play()` es un método estático. Java mira el tipo de la referencia para `rope2` y traduce la llamada como `Rope.play()`, entonces de acuerdo al análisis es correcto el inciso B y E.
12. En este ejercicio se pone en práctica identificar las variables locales, en donde en este fragmento de código actúan como constantes, dentro del método `feed` se declaran variables locales que es `monkey`, `giraffe`, `name`, a su vez dentro de un bloque pero afuera de este `if` se declara otra variable llamada `name`, desglosando el código en la línea 11, aunque está dentro de un `if`, está comparando si `0>0` como eso es falso el compilador en la línea 13 tiene un operador de post incremento que intenta modificar su valor. De acuerdo al análisis solo dos variables se mantienen intactas, la respuesta correcta es el inciso B.
13. En el fragmento de código se hace referencia cuando una variable es `static`, es importante recordar que cuando es así le pertenece a la clase, entonces en el fragmento donde se cambia el valor de variable de `rope 1` y `rope2` lo actualiza, como están declaradas dentro de la clase comparten la misma variable en memoria eso hace que lo que se imprima es el valor de 8, inciso D es el correcto.
14. Pide contabilizar cuantos errores hay en el código, antes de especificar cuantos errores hay, es vital entender el por qué, en línea 5 se ocupa `final` a la hora de declarar la variable de `name` y en línea 11, se está volviendo a declarar en otro método, recordando que esto no se puede hacer porque cuando se ocupa `final`, ya no deja modificar o volver a reasignar el valor a la variable declarada con `final`, aunque en el ejercicio le ponen el mismo valor, realmente ese no era el problema, después con `rightRope` están haciendo lo mismo, ya tiene `final`, se está volviendo a reasignarle valor, con esto van dos errores, ahora bien como en `main` se está utilizando `bench` es `static final`, como `final` solicita que tenga un valor guardado antes de que la clase empiece a correr, el método `main` tarde por lo que dura la

clase por lo tanto dejarla vacía en línea 4, es un error y el último error es justamente la línea 4 por estar vacía. En total son 4 errores, inciso D.

15. En este ejercicio se pide colocar que paqueteria es correcta para que compile de manera correcta en el método se está colocando un `ArrayList`, también `sort(list)`, este método pertenece a la clase utilitaria `java.util.Collections`, ahora bien para poder usar el método estático de otra clase sin escribir el nombre de la clase antes es obligatorio colocar `import static` y recordando que el `*` es que trae todo lo que está en ese paquete entonces la única opción correcta es inciso B, `import static java.util.Collections.*`.
16. Para entender este fragmento de código tuve que repasar el por qué se imprimen primero ciertos tipos de datos primero y luego por qué los otros, de primer momento es abordar el tema de sobrecarga progresiva y es cuando se pasa un argumento a un método sobrecargado, Java busca una coincidencia que puede ser exacta, widening, autoboxing y polimorfismo, ahora bien en el código en la línea 16 la variable `s` está declarada con `short` pero dentro del código no hay un método en la clase `Test` que imprima `short`, entonces por nivel de jerarquía se va a ir porque el que le quede más cercano, en este caso es `int`, después está mandando a imprimir un `boolean` pero como en el método no se está mandando a imprimir uno, se va a aplicar el Autoboxing para convertirlo en el objeto `boolean`, por lo tanto sería el método `object` el que se estaría imprimiendo y la última llamada es un `double` como dentro de la clase `Test` tampoco hay un método que mande a imprimir este tipo de dato, pero aquí como `double` es el tipo primitivo más grande no puede haber uno más arriba que él, entonces sería por herencia ya que la clase `double` es una subclase de `Object`, entonces se entrega por el método `object`, opción correcta es el inciso E, `int-Object-Object`.
17. En el siguiente fragmento de código se ve el paso por valor mezclado con la inferencia de tipos con `var`, en donde se está declarando que la variable le pertenece a la clase, ya que se está ocupando `static`, ahora bajo esta premisa estamos ocupando el paso por valor porque aunque se le cambia el valor a `x`, no se aplica ese cambio porque pasa el valor intacto, ahora bien se pide imprimir el valor más no el `result`, entonces respuesta correcta es el inciso B, 9.
18. Se vuelve a tocar en este ejercicio el paso por valor, entiendo entonces de acuerdo a lo que pide es que cuando se pasa un objeto a un método se pasa una copia de la dirección a la que apunta ese objeto por lo tanto `S1` se queda igual, sin embargo `S2` sí cambia ya que al pasar la variable local `b`, apunta al mismo objeto `s2`, de la memoria, entonces la línea de `b.append("b")` no cambia la dirección, sino que entra al objeto original y ahí le añade la `b`, por eso aquí si se modifica y es así como `s3` se queda con lo nuevo que es `a`, las respuestas correctas serían inciso A, D y F.
19. Se vuelve a ver final para este ejercicio y el bloque estático sólo se pudo ver lo que es `static`, es por eso que no puede tocar lo que son las variables de instancia, porque requieren que se cree el objeto primero, entonces analizando esto vemos que en la línea 6 lo que le hace falta es el `value2` que vale `purple`, en `value3` es igual a `Orange` y en línea 9 el valor ahora es `cyan`, `value2`, incisos correctos es el B, C y E.
20. Se vuelve a ver la sobrecarga de métodos y siguiendo el flujo del programa, en Java sigue un orden de prioridades, en la primera llamada solicita el entero, al encontrarlo imprime el 2-, en la segunda llamada le pasa un tipo `long`, como en el código no hay un método

long, se realiza el autoboxing, lo convierte el long a objeto Long y usa el polimorfismo por último para enviarlo al método que acepta un Object imprimiendo así el -4, por lo tanto opción correcta inciso A y F.

21. Se vuelve a tocar el tema de sobrecarga y antes de abordarlo es importante ver que para que una sobrecarga sea válida se debe de cumplir con dos condiciones, la primera que el método tenga el mismo nombre y la segunda es que tiene una lista de parámetros diferentes, y las otras pero para poder usar varargs es que un método sólo puede tener un único varargs y el segundo es el varargs deber ser obligatoriamente el último parámetro de la lista, ahora bien el nombre es igual a moo en el inciso B y la lista de parámetros es diferente, entonces este inciso es correcto y en el D, se puede observar lo mismo, así que las únicas opciones correctas es el inciso B y D.